

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

ЛАБОРАТОРНАЯ РАБОТА №2
ПО КУРСУ «ОБРАБОТКА И РАСПОЗНАВАНИЕ ИЗОБРАЖЕНИЙ»
ДЛЯ СТУДЕНТОВ 3-ГО КУРСА КАФЕДРЫ ММП
**«Изучение и освоение методов обработки и сегментации
изображений»**

Подготовил:
студент 317 группы
Морозов К. О.

Москва
2026

Содержание

1	Постановка задачи	2
2	Описание данных	2
3	Описание метода решения	2
3.1	Предобработка изображения	2
3.2	Отделение объектов от фона	3
3.3	Разделение объектов	3
3.4	Выделение признаков эталонных фишек	3
3.5	Формирование поворотов для произвольной фишки	4
3.6	Классификация	4
3.7	Визуализация меток классов	4
4	Описание программной реализации	5
5	Эксперименты	5
5.1	Построение признаковых описаний эталонных объектов	5
5.2	Работоспособность алгоритма	6
6	Выводы	7

1 Постановка задачи

В данной работе продолжается изучение способов обработки изображений на основе точечных и пространственных преобразований, а также изучаются способы генерации признаков описаний объектов на изображении. Применение различных методов обработки из классического CV рассматривается на задаче сегментации фишек из набора Тантрикс и классификации отдельных фишек и групп фишек на изображении. Задание заключается в разработке алгоритма обработки входного изображения, содержащего фишки, который решает одну из трёх задач: подсчёт числа объектов и анализ каждой отдельной фишки, классификация фишек и поиск замкнутого пути из фишек. Для удобной работы с алгоритмом реализуется приложение с возможностью загрузки произвольного изображения, визуализацией результатов работы алгоритма и возможностью сохранения полученных изображений.

2 Описание данных

В задании предоставляется некоторое число образцов (в том числе, в условии задания в файле pdf), на которых изображены одна фишка (файлы «Single_*.bmp»), несколько фишек, расположенных на небольшом расстоянии друг от друга (файлы «Group_*.bmp») и несколько фишек, расположенных вплотную (файлы «Path_*.bmp»). Все изображения характеризуются однотонным одинаковым фоном (бежевый цвет), на котором хорошо сегментируются объекты. Фишки – правильные шестиугольники чёрного цвета, на которых присутствуют 3 линии разных типов (короткая дуга большой кривизны, длинная дуга малой кривизны, прямолинейный сегмент) красного, жёлтого и синего цветов.

В задание входят задачи разных уровней сложности. Каждое заключается в решении конкретной задачи для определённых типов изображений (перечисленных выше):

- **Beginner:** подсчёт числа фишек на изображениях «Group_*.bmp» и определение типов линий на изображениях «Single_*.bmp»;
- **Intermediate:** классификация фишки на изображениях «Single_*.bmp» и классификация фишек с определением положения на изображениях «Group_*.bmp»;
- **Expert:** нахождение замкнутого маршрута и последовательности обхода фишек на изображениях «Path_*.bmp».

В данной работе представлено решение пункта **Intermediate** задания методами классического CV. В следующем пункте приводится описание решения.

3 Описание метода решения

После визуального изучения данных в пункте **Описание данных** сформирован план создания универсального алгоритма для решения поставленной задачи.

3.1 Предобработка изображения

Предобработка включает применение к входному изображению Гауссова фильтра.

Применение Гауссова фильтра приводит к сглаживанию текстур и шумов, связанных, в первую очередь, с освещением.

3.2 Отделение объектов от фона

Для отделения фона сначала определялись контура на изображении, закрашивались все замкнутые контура. Дополнительно на первом этапе для получения корректных результатов после выделения контуров использовалось несколько итераций морфологического закрытия (для замыкания всех нужных контуров). Для выделения контуров на изображении использовался детектор границ **Canny**. В данном пункте решения пользовались однотонностью фона (тем, что контура характеризуют объекты, на них наиболее резко меняется яркость пикселей).

3.3 Разделение объектов

На данном шаге из маски всех объектов получается маска для каждого объекта по отдельности. Для этого используется алгоритм **Watershed**.

На первом этапе для каждого пикселя в маске считается расстояние до фона. На следующем этапе анализируются локальные максимумы в полученной матрице. Если расстояние между 2-мя локальными максимумами достаточно большое, то это разные объекты, иначе один объект. В каждом объекте ставится маркер от 1 до числа полученных объектов, и запускается процесс «водораздела»: заполнение пикселей соответствующими маркерами. На выходе получаем матрицу, в которой пиксели с значением 0 соответствуют фону, 1 – первому объекту, 2 – второму, и так далее.

3.4 Выделение признаков эталонных фишек

После подробного изучения изображений всех фишек, возникла необходимость выделения признаков для представителя каждого класса (разных фишек) и их запоминания. В целом, алгоритм классификации будет заключаться в том, чтобы сравнить входную фишку с эталонными и найти среди них максимально похожую фишку, именно поэтому нам необходимо выделить признаки для эталонных представителей. Для этого, используется изображение, содержащее по одному представителю каждого класса (рис. 1): проходимся по каждому объекту, выделенному на прошлом шаге (считая за объекты только те, площадь которых в пикселях больше 500). Также делаем небольшой отступ от границ (с помощью эрозии), чтобы уменьшить влияние шума (сегментация не идеальна, может содержать части фона) и увеличить влияние самой фишки в итоговой работе алгоритма. Для каждого объекта (считаем уже за фишку) находим его `bbox`, вырезаем объект и делаем нормировку по каждому RGB-каналу (вычитается среднее, делим на дисперсию, нормировка необходима для устойчивости алгоритма к освещению). Затем, изображение приводится к заранее выбранному размеру (квадрат со стороной 200 пикселей) и нормируется (норма полученного объекта единица).

На выходе получаем признаковые описания (изображения размера $3 \times 200 \times 200$) для каждой из фишек (10).



Рис. 1: Изображение с эталонными фишками

Важным свойством здесь является ориентация каждой фишки: на рассматриваемом изображении все фишки расположены одним из углов строго вверх (одна из диагоналей, соединяющая самый верхний и нижний углы, находящиеся противоположно друг к другу, вертикальна), поэтому на следующих этапах необходимо будет приводить произвольную фишку к подобному виду, а также учитывать всевозможные повороты для корректного сравнения.

3.5 Формирование поворотов для произвольной фишки

Теперь работаем с произвольным изображением. Также выделяем объекты на нём (с тем же критерием по площади), Аналогично для каждого объекта делаем сначала эрозию, а затем вырезаем его bbox.

На следующем шаге нам необходимо расположить фишку на изображении так, чтобы один из её углов смотрел строго вверх (то есть одна из диагоналей шестиугольника (главных, между противоположными вершинами) была вертикальной. Для этого, изучаем маску: находим координаты самого верхнего и нижнего пикселей ($y_{\min}$ и $y_{\max}$). Затем, берём x , максимальный для $y_{\min}$, и x , минимальный для $y_{\max}$. Таким образом, получаем 2 точки, которые в случае правильного шестиугольника и идеального выделения его маски будут соответствовать двум противоположным углам. Остаётся только повернуть изображение относительно одной из точек на угол между диагональю (посчитанной между этими 2-мя точками) и вертикалью (аффинное преобразование).

Теперь надо сформировать всевозможные повороты изображения: для каждого угла угла в 0, 60, 120, 180, 240 и 300 градусов рассматриваем поворот полученного изображения на выбранный угол, делаем все действия, как и в случае эталонных изображений (выделение bbox, нормализация по каналам через среднее и дисперсию, ресайз и нормализация по норме тензора). На выходе для каждого объекта получаем по 6 представлений размера $3 \times 200 \times 200$.

3.6 Классификация

Классификация подразумевает собой сравнение всевозможных поворотов каждого объекта с представлениями эталонных. Для этого для каждого объекта считаем косинусное расстояние (или, в нашем случае, косинусный скор) между каждым поворотом и каждым эталонных объектов (используя полученные выше признаковые описания), и выбираем за результат классификации номер эталонного объекта с минимальным расстоянием (в нашем случае, с максимальным скором) до какого-нибудь из поворотов исходного объекта.

3.7 Визуализация меток классов

В случае работы с изображениями «Group_*.bmp» в уровне **Intermediate** необходимо также визуализировать метки классов для каждого объекта на изображении. Для определения наилучшей позиции для метки используется следующий алгоритм: для каждого объекта берутся координаты его bbox. Для каждого угла bbox смотрится расстояние (минимальное) до bbox других объектов (в дальнейшем будет сравниваться в том числе и с bbox меток) и находится тот угол, для которого полученное значение максимально. Таким образом, находим точку, которая будет являться центром метки. Далее строится bbox заранее выбранных размеров и маркеруется объект.

4 Описание программной реализации

Для реализации алгоритма использовался язык программирования **python**. Для осуществления работы пользователя в режиме диалога используется библиотека **tkinter**, с помощью которой реализуется приложения-обёртки над алгоритмом. Код алгоритма находится в файле **processing.py**, код приложения в файле **app.py**. Необходимый список библиотек представлен в файле **requirements.txt**. Для запуска самой программы необходимо запустить файл **app.py**.

На этапе предобработки (функция **first_preprocessing**) использовалась библиотека **cv2** и функция из неё: Гауссов блюр с ядром 5 и автоматическим расчётом параметра стандартного отклонения.

На этапе отделения объектов от фона (функции **get_union_objects_contours** и **get_union_objects_mask**) использовались библиотеки **cv2** и **numpy**. Для построения маски контуров (функция **get_union_objects_contours**) использовались детектор границ **Canny** (на чёрно-белом изображении ищет те пиксели, где яркость меняется наиболее резко и относит их к контурам), и морфологическое замыкание (для замыкания контуров, что важно для их заливки). В функции **get_union_objects_mask** происходит поиск всех замкнутых контуров и их заливка с помощью функций **findContours** и **drawContours**.

На этапе разделения объектов (функция **get_object_masks**) используется библиотека **skimage** и реализация всех методов из неё (с использованием, в том числе, библиотеки **cv2**), в том числе алгоритм **watershed**.

Этап поиска поворотов для каждой фишки (функция **get_objects_with_all_turn**) происходит, сначала, обработка маски (подсчёт площади, эрозия) с помощью стандартных функций **numpy** и **cv2**, затем происходит подсчёт углов поворота и нахождение точек, относительно которых совершается поворот (через геометрические свойства), далее совершается аффинное преобразование (с помощью стандартной функции библиотеки **cv2**) и происходит обработка объекта на нём: вырезание **bbox**, нормализация по каналам (через среднее и дисперсию), приведение изображения к размерам 200×200 , нормализация (весь тензор признаков размера $3 \times 200 \times 200$ нормализуется по евклидовой норме). Для этих шагов используются вспомогательные функции **get_turn**, **get_bbox**, **get_crop_image** и **normalize_mask**, реализованные с помощью **numpy** и **cv2**, и стандартные функции из этих библиотек.

На этапе классификации (функция **classification**) используются стандартные методы работы с **numpy**-массивами: вытягивание в векторы, перемножение, поиск максимумов и аргументов максимума по определённым измерениям.

Этап визуализации меток на изображении реализован в функции **visualisation_predict_on_image**. На нём получаем **bbox** объектов с помощью ранее реализованной функции **get_bbox**, находим точку для размещения метки для каждого объекта методами из библиотеки **numpy** и визуализируем их на изображении с помощью функций из **cv2**.

5 Эксперименты

Результаты всех экспериментов приведены в ноутбуке **LAB2_Experiments.ipynb**.

5.1 Построение признаковых описаний эталонных объектов

Для данного пункта была реализована функция **get_objects**, которая вычисляла признаковое представление объектов на входном изображении (без поворотов). Таким образом, получили **numpy**-массив размера $10 \times 200 \times 200 \times 3$ – признаковое описание эталонных объектов. Он сохраняется в файл «**all.npy**». Также в файле с экспериментами и на рисунке 2 представлены визуализации признаковых

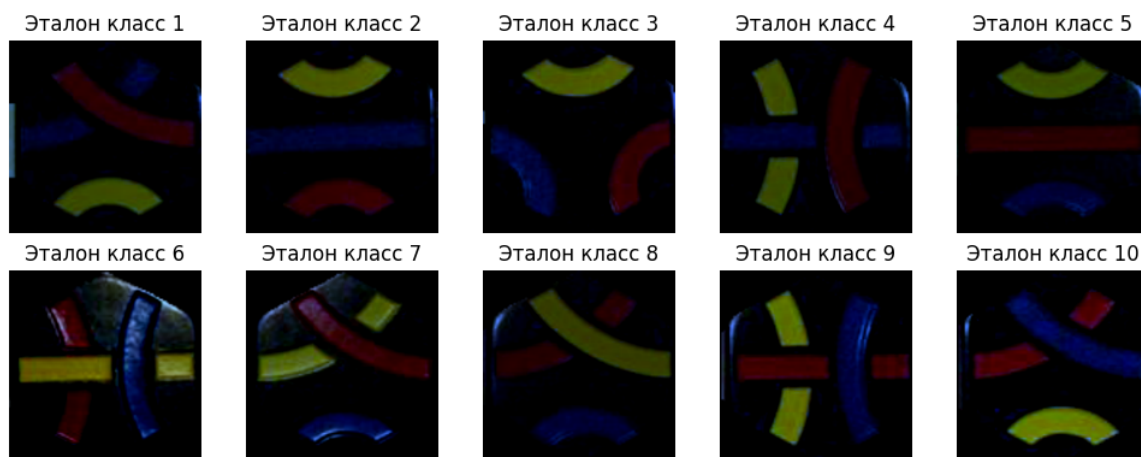


Рис. 2: Визуализация признаков описаний эталонных фишек

представлений эталонных фишек. По ним видно, что bbox фишек определяются корректно, а также нормализация по каналам (вычитание среднего и деление на дисперсию) действительно уменьшают влияние освещения на полученные признаки.

5.2 Работоспособность алгоритма

Работоспособность алгоритма проверена на данных для отладки и обучения алгоритма изображениях типа «Single_*.bmp» и «Group_*.bmp» из архива «Обучение.rar», а также некоторых изображениях из pdf-файла с условием задания. По итогу алгоритм дал **100%** точность классификации в задаче: все объекты были верно обнаружены и всем объектам были даны правильные номера фишек. Таким образом, в данной работе удалось успешно справиться с заданием уровня **Intermediate**. Результат работы алгоритма на примерах обучающих данных представлен на рисунке 3.

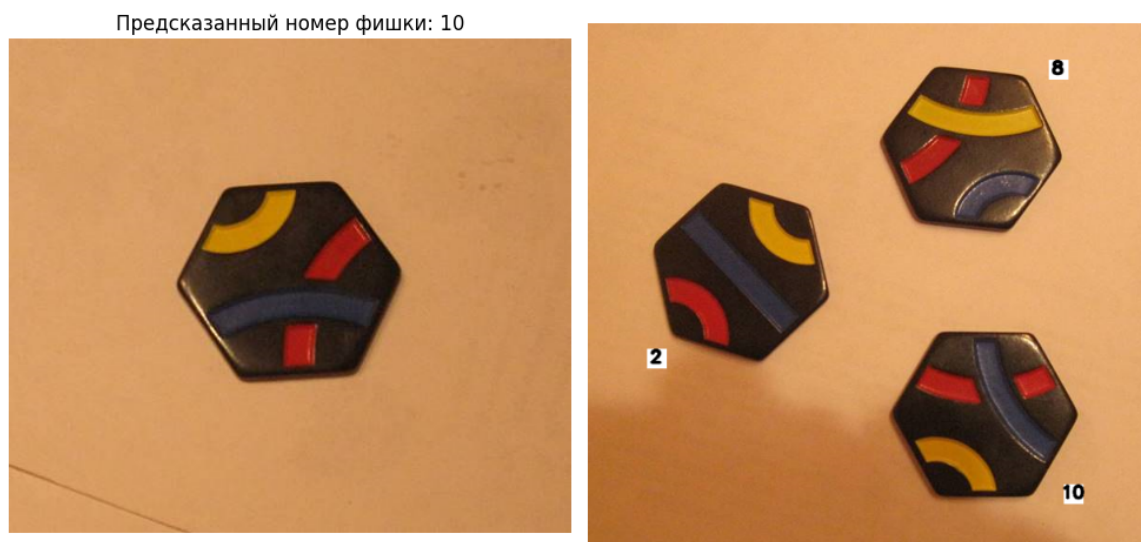


Рис. 3: Работа алгоритма на картинке типа «Single_*.bmp» и «Group_*.bmp» из образцов

6 Выводы

В заключение, в данной работе были изучены способы генерации признаков описаний объектов в классическом **CV** и реализован алгоритм, основывающийся на этих методах, а также точечных и пространственных преобразованиях изображений, для решения задачи определения номеров фишек из набора Тантрикс. Данный алгоритм, использующий исключительно методы классического **CV**, показал **100%** точность классификации на обучающих данных. Таким образом, в работе успешно решена задача уровня **Intermediate** (и, следовательно, **Beginner**).